UNIVERSITÉ ABDELMALEK ESSAÂDI
École Nationale des Sciences Appliquées d'Al Hoceima

COMPTE RENDU DE TRAVAUX PRATIQUES :

TP N° 3 – Java SE

Module : Programmation orientée objet java

Manipulation des notions de POO :

objets , classes , constructeurs , encapsulation , this , final , static …

Objectif

Comprendre et appliquer les concepts fondamentaux de la programmation orientée objet en Java à travers des exercices pratiques.

Réalisé par : Aya MAJJATI

Encadré par : Pr. Mohamed CHERRADI

Filiére : Ingénieurie des Données 1

Année universitaire : 2025 – 2026

Exercice 1 : Point

Dans cet exercice nous modéliserons un point du plan :

Attributs : x et y qui représente les coordonnées du point , de type double , déclarées en **private** .

Méthodes :

- Afficher() :pour afficher le point .
- Déplacer() : pour déplacer le point à un autre .
- CalculeDistanceOrigine() : methode de type double , qui calcule la distance du point à l'origine et retourne cette distance par l'expression mathématique convenable .

J'ai ajouté des méthodes supplémentaires :

- DistancePoint() : méthode de type double , prend en paramètre l'objet Point , pour calculer la distance entre deux points (le point qu'on a et le point déclarer dans les paramètres) , par l'expression mathématiques .
- getX() et getY() : vue que les attributs sont déclarés en private , il faut implémenter les Getters .

Constructeur : on a deux types de constructeurs :

- **Sans paramètres :** initialiser par des valeurs par defaut .
- **Avec paramètres :** permet d'initialiser directement les coordonnées du point **.**

<u>Code source :</u>

```java
package TP3;
class Point {
    private double x , y ;

    public Point () {
        this.x=1;
        this.y=1;
    }
    public Point (double x , double y){
        this.x=x;
        this.y=y;
    }
    public void afficher() {
        System.out.println("le point ("+x+","+y+")");
    }
    public void deplacer(double dx , double dy) {
        x = x + dx;
        y = y + dy;
    }
    public double calculeDistanceOrigine() {
        return  Math.sqrt(x*x+y*y);
    }
    public double DistancePoint (Point p) {
        return Math.sqrt(Math.pow(p.x - x, 2)+Math.pow(p.y - y, 2));
    }
    public double getX() {
        return x;
    }
    public double getY() {
        return y;
    }
}
```

Test dans le main() :

```java
ic class Ex1 {
public static void main (String[] args) {
    Point p1 = new Point(3,4);
    p1.afficher();
    System.out.println("Distance à l'origine : " + p1.calculeDistanceOrigine());
    p1.deplacer(2, 1);
    System.out.print("Après déplacement de (2,1) -> ");
    p1.afficher();
    Point p2 = new Point(2,2);
    System.out.println("Distance au point :("+ p2.getX() +","+ p2.getY()+") est :"
    +p1.DistancePoint(p2));

}
```

Résultat console :

```
<terminated> Ex1 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jr
le point (3.0,4.0)
Distance à l'origine : 5.0
Après déplacement de (2,1) -> le point (5.0,5.0)
Distance au point :(2.0,2.0) est :4.242640687119285
```

Exercice 2 : Rectangle

Dans cet exercice , nous modéliserons un rectangle , avec :

Attributs : largeur , hauteur ., de type double , déclarés en **private**.

Constructeur : nous implémenterons un constructeur avec paramètres qui permettra d'initialiser directement les dimensions du rectangle .

Les méthodes : nous créons deux méthodes de calcul :

- surface () qui retourne le produit de la largeur par la hauteur, et
- périmètre() qui retourne la somme des cotés (2*(largeur+hauteur)**) .**

Code source :

```java
package TP3;
class Rectangle {

    private double largeur , hauteur ;

    public Rectangle(double l , double h ) {
        this.largeur=l ;
        this.hauteur=h ;
    }
    public double surface () {
        return largeur * hauteur;
    }
    public double perimetre() {
        return 2 * (largeur + hauteur);
    }
```

J'ai ajouté une fonction afficher() , pour afficher les résultats obtenus et faire un test dans le main() .

```java
    public void afficher() {
        System.out.println("la hauteur : "+hauteur);
        System.out.println("la largeur  : "+largeur);
        System.out.println("le perimetre  : "+ perimetre());
        System.out.println("la surface   : " + surface());
    }

}
public class Ex2 {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(2,4);
        r.afficher();
    }
}
```

Résultat console :

```
<terminated> Ex2 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspo
la hauteur : 4.0
la largeur  : 2.0
le perimetre  : 12.0
la surface   : 8.0
```

Exercice 3 : Etudiant

Dans cet exercice , nous modeliserons une classe Etudiant avec :

- **Attributs :** cne (Code National de l'Étudiant) de type String et note de type double. Ces attributs sont **privés** .
- **Constructeur :** Un constructeur avec deux paramètres (cne, note) est défini pour initialiser l'objet dès sa création en utilisant le mot-clé **this**.
- **Méthodes :**
- toString() : Cette méthode permet de retourner une chaîne de caractères formatée contenant les informations de l'étudiant.
- estAdmis() : C'est une méthode qui retourne un **booléen** (true si la note est supérieure ou égale à 10, false sinon).

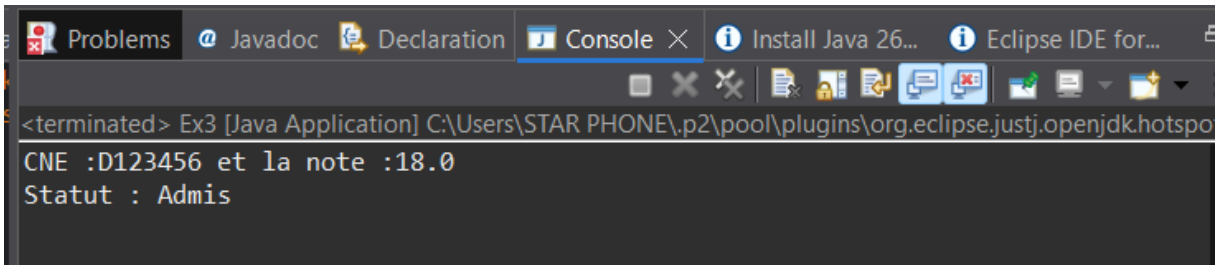
Code source :

```java
package TP3;
class Etudiant {
    private String cne ;
    private double note ;

    public Etudiant(String cne , double note) {
        this.cne=cne;
        this.note=note;
    }
    public String toString() {
        String S = "CNE :"+cne + " et la note :"+note ;
        return S ;
    }
    public boolean estAdmis() {
        return note >= 10 ;
    }
}
```

Test dans le main():

```java
public class Ex3 {
    public static void main(String[] args) {
        Etudiant e1 = new Etudiant("D123456", 18.0);
        System.out.println(e1.toString());

        if(e1.estAdmis()) {
            System.out.println("Statut : Admis");
        } else {
            System.out.println("Statut : Ajourné");
        }
    }
}
```

Résultat console :



Exercice 4 : NombreComplexe

Dans cet exercice nous modéliserons un nombre complexe :

Attributs : réel et imaginaire qui représente les parties réel et imaginaires **,** ils sont de types double , déclarés en private .

Méthodes :

- Addition() : de type NombreComplexe , prend en paramètres un objet NombreComplexe et calcule l'addition avec l'objet courant .
- Multiplication() : de type NombreComplexe , prend en paramètres un objet NombreComplexe et calcule la multiplication avec l'objet courant .
- Afficher() : méthode affiche le nombre complexe .

Constructeur : NombreComplexe() avec paramètres , pour initialiser l'objet .

Code source :

```
1  package TP3;
2  class NombreComplexe{
3      private double reel , imaginaire ;
4
5      public NombreComplexe(double reel , double imaginaire) {
6          this.imaginaire=imaginaire;
7          this.reel=reel;
8      }
9      public NombreComplexe addition(NombreComplexe z) {
10         double r = this.reel+z.reel ;
11         double i = this.imaginaire+z.imaginaire ;
12         return new NombreComplexe(r, i);
13     }
14     public NombreComplexe multiplication(NombreComplexe z) {
15         double r = (this.reel * z.reel) - (this.imaginaire * z.imaginaire);
16         double i = (this.reel * z.imaginaire) + (this.imaginaire * z.reel);
17         return new NombreComplexe(r, i);
18     }
19     public void afficher() {
20         System.out.println("  "+reel+"+"+imaginaire+" i ");
21     }
22 }
```

main() :

```
24 public class Ex4 {
25     public static void main(String[] args) {
26         NombreComplexe z1 = new NombreComplexe( 1 ,2 );
27         z1.afficher();
28         NombreComplexe z2= new NombreComplexe( 2 ,3 );
29         NombreComplexe som = z1.addition(z2);
30         System.out.println("la somme est : ");
31         som.afficher();
32         NombreComplexe mul = z1.multiplication(z2);
33         System.out.println("la multiplication est : ");
34         mul.afficher();
35     }
```

Résultat console :

```
<terminated> Ex4 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspo
  1.0+2.0 i
la somme est :
  3.0+5.0 i
la multiplication est :
  -4.0+7.0 i
```

Exercice 5 : CompteBancaire

Dans cet exercice nous modéliserons un compte bancaire **.**

Attributs : numéro (numéro du compte) du type int et le solde de type double déclaré en private .

Méthodes : tout les méthodes sont de type void :

- Deposer() : permet de de déposer un montant qui le prend en paramètre .
- Retire() : permet de de retirer un montant qui le prend en paramètre .
- Transférer() : permet de de déposer un montant à un destinataire de type CompteBancaire , prend en paramètre le montant et l'objet .
- afficherSolde() : affiche le solde du compte bancaire .

Constructeur : CompteBancaire() avec des paramètres pour initialiser l'objet .

<u>Code Source :</u>

```
1  package TP3;
2
3  class CompteBancaire{
4      private int numero ;
5      private double solde ;
6      public CompteBancaire(int numero , int solde ) {
7          this.numero=numero ;
8          this.solde=solde;
9      }
10     public void deposer(double montant) {
11          if (montant > 0) {
12              this.solde += montant;
13              System.out.println("Vous avez posé :"+montant);
14          }
15     }
16     public void retirer(double montant) {
17         if (montant <= solde && this.solde >= montant) {
18             this.solde -= montant ;
19             System.out.println("Vous avez retiré :"+montant+" ,votre Solde est : "
20         }else {
21             System.out.println("retirage impossible");
22         }
23     }
24     public void transferer(double montant, CompteBancaire X) {
25         if(montant > 0) {
26             this.retirer(montant);
27             X.deposer(montant);
28             System.out.println("Vous avez transferé "+montant+" vers le compte de r
29         }else {
30             System.out.println("transfert impossible");
31         }
32     }
33     public void afficherSolde() {
34         System.out.println("\nVotre solde est : "+this.solde);
35     }
36 }
```

<u>main() :</u>

```java
public class Ex5 {
    public static void main(String[] args) {
        CompteBancaire C1= new CompteBancaire(1234 ,5000);
        CompteBancaire C2= new CompteBancaire(5678 ,9000);
        C1.afficherSolde();
        C2.afficherSolde();

        C1.transferer(1500, C2);

        C1.afficherSolde();
        C2.afficherSolde();

        C1.retirer(2000);
        C1.afficherSolde();

        C2.deposer(3000);
        C2.afficherSolde();

    }

}
```

Résultat console :

```
<terminated> Ex5 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspo

Votre solde est : 5000.0

Votre solde est : 9000.0
Vous avez retiré :1500.0 ,votre Solde est : 3500.0
Vous avez posé :1500.0
Vous avez transferé 1500.0 vers le compte de numero :5678

Votre solde est : 3500.0

Votre solde est : 10500.0
Vous avez retiré :2000.0 ,votre Solde est : 1500.0

Votre solde est : 1500.0
Vous avez posé :3000.0

Votre solde est : 13500.0
```

Exercice 6 : Voiture

Dans cet exercice nous modéliserons une voiture :

Attributs : marque qui represente la marque de la voiture , de type String , et vitesse qui représente la vitesse de la voiture de type int , déclaré en private .

Méthodes : de type void :

- **Accélérer() :** permet d'accelerer la vitesse en ajoutant la vitesse déclaré en parametre à la vitesse de l'objet(Voiture) , à condition que cette somme ne dépasse pas 240km/h.
- **Freiner() :** permet de freiner la vitesse en faisant la soustraction de la vitesse de l'objet et la vitesse a freiner à condition .

Constructeur : Voiture() avec des paramètres pour initialiser l'objet

Code source :

```
package TP3;

class Voiture{
    private String marque ;
    private int vitesse;
    public Voiture(String marque , int vitesse) {
        this.marque=marque;
        this.vitesse=vitesse;
    }
    public void accelerer(int vitesse) {
        if (vitesse + this.vitesse <= 240) {
            this.vitesse +=vitesse;
            System.out.println("Accélération de "+vitesse+" km/h...");
        }else {
            System.out.println("Vous avez depassé la vitesse maximale (240km/h) !!");
        }
    }
    public void freiner(int vitesse) {
        if(this.vitesse > vitesse ) {
            this.vitesse -= vitesse;
        }else {
            this.vitesse = 0;
            System.out.println("la voiture est arretée");
        }
    }
}
```

main() :

```
public class Ex6 {
    public static void main(String[] args) {
        Voiture V = new Voiture("mercedec", 90);
        V.accelerer(30);
        V.accelerer(200);
        V.freiner(20);
        V.freiner(100);
    }
}
```

Résultat console :

```
terminated> Ex6 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.ope
Accélération de 30 km/h...
Vous avez depassé la vitesse maximale (240km/h) !!
la voiture est arretée
```

Exercice 7 : Produit

Dans cet exercice, nous modéliserons un système de gestion de produits (stock) :

Attributs :

- id, nom, description, prix et quantite représentant les caractéristiques d'un produit. Ils sont de types int, String et double, déclarés en **private**.
- **Stockage statique :** Un tableau Produits de taille 100 et un entier nbProduits (déclarés en static) pour mémoriser et gérer l'ensemble des objets créés.

Méthodes :

- **ajouterProd() :** Ajoute l'objet courant (this) dans le tableau statique et incrémente le compteur.
- **modifierProd() :** Méthode statique qui recherche un produit par son ID et met à jour son nom et son prix.
- **supprimerProd() :** Méthode statique qui recherche un produit par son ID, le supprime du tableau et décale les éléments suivants pour maintenir la liste à jour.
- **getProduitByID() :** Méthode statique qui affiche les informations d'un produit spécifique à partir de son identifiant.
- **getALLProduit() :** Méthode statique qui parcourt le tableau et affiche la liste de tous les produits enregistrés.

Constructeurs : Produit() avec paramètres , pour initialiser les attributs de l'objet .

<u>Extrait du Code Source :</u>

```
3  class Produit{
4      private int id;
5      private String nom , description;
6      private double prix ;
7      private int quantite;
8
9       private static Produit[] Produits = new Produit[100];
10      private static int nbProduits = 0;
11
12     public Produit(int id , String nom , String description , double prix , int quantite)
13         this.id=id;
14         this.nom=nom;
15         this.description=description;
16         this.prix=prix;
17         this.quantite=quantite;
18     }
19     public void ajouterProd() {
20         Produits[nbProduits]=this;
21         nbProduits++;
22         System.out.println(" le produit "+this.nom+"a été ajouté au système ");
23     }
24     public static  void modifierProd(int id,String nv_nom ,double nv_prix) {
25         for(int i=0; i < nbProduits ; i++) {
26             if(Produits[i].id == id) {
27                 Produits[i].nom= nv_nom;
28                 Produits[i].prix= nv_prix;
29             }
```

Main() :

```java
public class Ex7 {
    public static void main(String[] args) {
        Produit p1 = new Produit(1, "Écran LED", "24 pouces HD", 1200.0, 5);
        Produit p2 = new Produit(2, "Clavier", "Mécanique RGB", 350.0, 10);
        Produit p3 = new Produit(3, "Souris", "Sans fil", 150.0, 8);
        p1.ajouterProd();
        p2.ajouterProd();
        p3.ajouterProd();

        Produit.getALLProduit();
        Produit.modifierProd(2, "Clavier Sans Fil", 400.0);
        Produit.supprimerProd(1);
    }
}
```

Résultat console :

```
<terminated> Ex7 [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
 le produit Écran LEDa été ajouté au système
 le produit Claviera été ajouté au système
 le produit Sourisa été ajouté au système
Liste des produits
ID: 1 | Nom: Écran LED | Prix: 1200.01200.0 | Quantité: 5
ID: 2 | Nom: Clavier | Prix: 350.0350.0 | Quantité: 10
ID: 3 | Nom: Souris | Prix: 150.0150.0 | Quantité: 8
Produit supprimé .
```

Exercice – Bonus | Deviner un nombre aléatoire :

Cet exercice bonus consiste à réaliser un jeu interactif entre l'ordinateur et l'utilisateur.

- Le programme utilise la classe Random pour générer un entier compris entre 1 et 10.
- La saisie est gérée par la classe Scanner. Le programme utilise une boucle while pour permettre à l'utilisateur de proposer plusieurs essais jusqu'à trouver la bonne réponse.
- À chaque essai, des messages (« Plus grand » ou « Plus petit ») orientent l'utilisateur vers la solution.
- Pour valoriser la rapidité de l'utilisateur, un score est calculé en fonction du nombre de tentatives effectuées. Nous avons utilisé la formule Score = 10 - nombre d'essais.

Code source :

```java
package TP3;
import java.util.*;
public class Ex7Bonus {
    public static void main  (String [] args ) {
        Scanner sc = new Scanner(System.in);
        Random rand = new Random();
        int n = rand.nextInt(10)+1;
        int saisi = 0 ;
        int essais =0;
        System.out.println("Devinez le nombre entre 1 et 10 :");
        while (saisi != n) {
            System.out.print("Votre essai : ");
            saisi = sc.nextInt();
            essais++;

            if (saisi < n)
                System.out.println("Plus grand ");
            else if (saisi > n)
                System.out.println("Plus petit ");
            else {
                System.out.println("Bravo !");
                int score = 10 - essais;
                if (score < 0) score = 0;
                System.out.println("Tentatives : " + essais);
                System.out.println("Votre score : " + score + " / 10");
            }
        }
        sc.close();
    }
}
```

Résultat console :

```
<terminated> Ex7Bonus [Java Application] C:\Users\STAR PHONE\.p2\pool\plugins\org.eclip
Devinez le nombre entre 1 et 10 :
Votre essai : 6
Plus petit
Votre essai : 4
Bravo !
Tentatives : 2
Votre score : 8 / 10
```